These methods are intended to control the behavior of the web server, forming an overarching agreement between the client and server about what effects on the target resource the request will have. Some methods – such as `GET` or `HEAD` – do not change the resource: these are read-only methods used only to fetch data. Other methods are intended to cause some change to the resource: `PUT` creates the resource, `POST` is intended to trigger some processing or partial change, and of course `DELETE` is for removing the resource. There is also an HTTP extension method, `PATCH`, which is not part of the RFC 9110 standard (it is instead proposed in RFC 5789) but is supported by `cli-server`. `PATCH` is intended for "partial updates," and such a request would only contain the data required for the partial update. It is quite common to see `POST` requests doing the job of `PATCH`.

It's about time we saw some of this working within the context of PHP, so let's launch a web server and make a request so we can see how to interact with request data. PHP's `cli-server` SAPI is a good choice here; after all, its intended use is for quick-and-dirty debugging and development. Way back in Chapter 1, "Getting Started", we saw how to run a Dockerized PHP web server, but here's a quick recap. Open two command line terminals, and make sure the current directory is the same for both (you can check with the `$ pwd` command on just about any *nix-like system). In the first one, run the following command:

```
$ docker run -it --rm \
  -p 8080:8080 \
  -v "$PWD":/usr/src/myapp \
  -w /usr/src/myapp \
  php:8.4-cli \
  php -S 0.0.0.0:8080
```

After the docker output (potentially fetching the `php:8.4-cli` image if it isn't found on your system), you should see the following line…

```
PHP 8.4.6 Development Server (http://0.0.0.0:8080) started
```

… which means the PHP cli binary is bound to TCP 8080 and ready to accept requests from any IP address. Leave this terminal window running, and in another, create a file called `index.php`. This file needs to exist where the PHP web server was run (as per the `-v "$PWD":/usr/src/myapp` command line option). The contents of the file should be…